

Sentence Embeddings - MNLP HW1

Antonio Serra 1893530 - Daniele Falanga 1918515

1 Introduction

This report presents a dense retrieval system for semantic search in a question-answering setting, focusing on retrieving the most relevant supporting passage rather than generating an answer. Given a natural-language query and a set of candidate chunks, the goal is to rank the candidates so that the correct chunk is retrieved as high as possible. We start from the two required baselines, `distilbert-base-uncased` and `all-MiniLM-L6-v2`, and then study the effect of supervised fine-tuning on retrieval quality.

Beyond the base requirements, we also consider additional analyses. First, we report Mean Reciprocal Rank (MRR) as an additional retrieval metric. Second, we explore an additional model, TinyBERT, to assess whether a lighter architecture can remain competitive after task-specific fine-tuning. Our main objective is to compare zero-shot and fine-tuned bi-encoder systems, discussing both performance gains and the trade-off between model quality and architectural simplicity.

2 Methodology

Our system follows a bi-encoder retrieval pipeline. Each query and each candidate chunk are encoded independently into dense vector representations using the same transformer encoder. Sentence-level embeddings are obtained through mean pooling over the token representations, masked by the attention mask so that padding tokens do not affect the final embedding. Once the query embedding and the candidate embeddings are computed, the candidates are ranked by cosine similarity.

For the base requirement, we implemented two baseline systems. The first baseline uses `distilbert-base-uncased` as a generic encoder without sentence-level fine-tuning. The second baseline uses `all-MiniLM-L6-v2`, which is already optimized for sentence similarity tasks and is there-

fore expected to be stronger in retrieval. In both cases, ranking is performed over the full set of candidate chunks for each query.

Evaluation is based on Hit@1, Hit@3, and Hit@5, implemented from scratch. For each query, Hit@k assigns value 1 if the gold chunk appears within the top- k retrieved candidates and 0 otherwise; the final score is the average over the test set. As an additional requirement, we also compute Mean Reciprocal Rank (MRR), which rewards systems that place the correct chunk in higher positions even when it is not ranked first.

For fine-tuning, we adopt a triplet-based supervised strategy. For each training example, we construct a triplet where the query acts as the anchor, the gold chunk as the positive example, and the negative example is selected from the remaining candidates. We experiment with two negative sampling strategies: (i) **random sampling**, where the negative is drawn uniformly from incorrect candidates, and (ii) **hard negative mining**, where the most similar incorrect chunk is selected according to the current model. While hard negatives are expected to provide a stronger training signal, we empirically observe that random sampling yields better performance and significantly reduces training time, since it avoids additional encoding steps required for hard negative selection. This can be explained by the presence of semantically similar but partially relevant chunks, which may introduce noise when treated as negatives. As a result, random negatives provide a more stable supervision signal and are used in our final configuration.

The optimization objective is based on cosine similarity, encouraging high similarity between the anchor and the positive example, and low similarity between the anchor and the hard negative. This formulation allows the model to progressively refine the embedding space by separating fine-grained semantic distinctions that are crucial for retrieval performance.

In addition to the mandatory fine-tuning of DistilBERT, we fine-tune MiniLM with the same training logic for comparison, and we extend the analysis with huawei-noah/TinyBERT_General_4L_312D as an additional model. This allows us to compare three architectures under the same retrieval framework and training protocol.

3 Experimental Setup

We compare models of different sizes, measured in terms of number of parameters: DistilBERT (66M parameters), MiniLM (22M), and TinyBERT (14M). Experiments are run on a Google Colab NVIDIA T4 GPU. Training times vary across models, with DistilBERT (40 min/epoch), MiniLM (15 min/epoch), and TinyBERT (12 min/epoch) when using hard negative mining. When using random negative sampling, training is significantly faster (around 6 minutes per epoch), as it does not require additional forward passes to compute similarities for negative selection.

For the baseline experiments, we evaluate `distilbert-base-uncased` and `all-MiniLM-L6-v2` in a zero-shot setting. Queries and candidate chunks are tokenized with the corresponding model tokenizer and encoded with the transformer backbone. Embeddings are produced by mean pooling, and cosine similarity is used for ranking. For MiniLM, we also verify the baseline through the native `SentenceTransformer` interface, which reproduces **exactly** the official target scores.

For supervised training, we use a batch size of 32, a learning rate of 2×10^{-5} , and 1 epoch for all models, as increasing the number of epochs did not yield significant performance gains. The final training loss is 0.1196 for DistilBERT and 0.0979 for MiniLM. The optimizer is AdamW. For TinyBERT, we adopt the same training setup but use a smaller learning rate of 1×10^{-5} for stability, achieving a final training loss of 0.1900. During training, each batch contains anchor, positive, and negative texts; negatives are selected with hard negative mining. After training, models are evaluated with the same retrieval procedure used for the baselines, ensuring a fair comparison.

4 Results

Table 1 reports the main results on the test split. The DistilBERT baseline reproduces the required reference scores exactly, confirming the correctness

of the implementation. For MiniLM, the native `SentenceTransformer` evaluation matches the expected baseline, while the plain `AutoModel` variant yields nearly identical values.

Model	H@1	H@3	H@5	MRR
DistilBERT (Base)	0.1765	0.4435	0.6045	0.3657
MiniLM (Base)	0.4495	0.7195	0.8100	0.6104
MiniLM-ST (Base)	0.4480	0.7165	0.8080	0.6093
DistilBERT (FT)	0.5565	0.8025	0.8805	0.6978
MiniLM (FT)	0.5905	0.8400	0.9145	0.7277
TinyBERT (Base)	0.0765	0.2125	0.3250	0.2098
TinyBERT (FT)	0.4715	0.7090	0.8025	0.6167

Table 1: Main retrieval results on the test split.

The most important finding is that supervised fine-tuning produces a very large improvement over the baselines. DistilBERT improves from 17.65% to 55.65% in Hit@1, with similarly strong gains in Hit@3, Hit@5, and MRR. This confirms that a generic encoder is not sufficient for this retrieval task in zero-shot mode, but can become highly effective once adapted to the homework distribution.

MiniLM starts from a much stronger baseline, which is expected because it is already tuned for sentence-level similarity. However, fine-tuning still yields a clear improvement, bringing Hit@1 to 59.05% and MRR to 0.7277. Interestingly, after fine-tuning, DistilBERT and MiniLM reach almost the same final performance. This suggests that task-specific supervision is strong enough to reduce much of the initial gap between a generic encoder and an off-the-shelf sentence embedding model.

TinyBERT provides an informative additional comparison. In zero-shot mode, it performs poorly, which indicates that its compact architecture is not immediately suitable for this retrieval setting. After fine-tuning, however, it improves substantially, reaching 47.15% Hit@1 and 0.6167 MRR. Although it remains below the two stronger fine-tuned models, the gain is large enough to show that lightweight models can still become useful when trained on the target task. This makes TinyBERT an interesting option when computational cost is a stronger constraint.

Overall, the results show that fine-tuning plays a crucial role in improving retrieval performance on the target dataset.

5 Appendix

This appendix includes a histogram that provides a visual summary of all the results of Hit 1 and MRR, enabling a quicker and more intuitive analysis.

